Project 4: Method & User Study Summary

Recommender Systems and Active Learning

Table of Contents

Project Overview: Influence of Future Predictions 1
Task 1: Guided Active Learning 1
Task 2: User Study Design and Evaluation 1
Task 3: Study Interface Implementation 1

Project Overview: Influence of Future Predictions on Active Learning in Recommender Systems

- This project addresses the cold-start problem in recommender systems, where new users have no prior ratings and the system must quickly learn their preferences.
- We use the MovieLens dataset (100k ratings, 9k movies, 600 users) to build a movie recommender that adapts to new users by interactively eliciting their tastes.
- Users rate movies on a scale from 0.5 to 5, indicating either their actual rating or their interest level if they haven't seen the movie.
- Unlike standard active learning, our method provides users with real-time feedback on how each rating will affect future recommendations, encouraging strategic and informed responses.

Task 1: Guided Active Learning for Cold-Start Recommendation

- Matrix factorization is performed using Singular Value Decomposition (SVD) on the user-item rating matrix, with K=50 latent factors (see model_training.py).
- Collaborative filtering is used: user (U) and movie (V) matrices are learned to minimize the error between predicted and actual ratings, with regularization applied during SVD.
- For new users, their latent representation is initialized and updated as they provide ratings, using the pre-trained movie factors (V) and regularized least squares (see recommender.py).
- Genre information from movies.csv is available for recommendations, but tags.csv is not actively used in the current implementation.
- Movie metadata (movie_metadata.json) is used to display genre and similar movies in the interface.
- Backend logic in recommender.py and views.py handles rating submissions, updates user factors, and generates recommendations in real time.
- The interface (study_interface.html, script.js) allows users to rate movies and see updated recommendations, but does not visualize the impact of each rating in detail.
- Data processing, model training, and prediction logic are implemented in Python and Django, with numpy used for matrix operations and storage (user_factors.npy, movie_factors.npy).

Task 2: User Study Design and Evaluation

- The study interface allows users to rate movies and records their ratings for later analysis.
- Metrics such as rating diversity and completion rate can be inferred from the ratings data exported from the system.

Task 3: Study Interface Implementation

- The landing page (index.html) introduces the study, provides access to documentation, and allows users to start the study.
- The rating interface (study_interface.html) presents movies with genre information and allows users to submit ratings.
- The interface is responsive and provides clear instructions for users.
- All interface logic is implemented using Django templates, static CSS/JS, and backend views for integration and user experience.